

Universidad del Valle de Guatemala

Facultad de Ingeniería

Carrera de Ciencia de la Computación y Tecnologías de la Información

Ejercicio 3

Programación orientada a objetos

Análisis

Clase	Propiedades	Métodos
Main		• <code>public static void main(String[] args):</code> Es el módulo principal desde donde se llamará el inicializador.
OrdenServicio	• <code>Private int numOrden:</code> Identificador de la orden. • <code>Private String propietario:</code> Nombre del propietario. • <code>Private String placa:</code> Placa del vehiculo. • <code>Private String descripcion:</code> Descripción breve de lo que se realizara en el servicio. • <code>Private double costo:</code> Costo estimado del servicio.	• <code>Public OrdenServicio(int numOrden, String propietario, String placa, String descripcion, double costo):</code> Para inicializar los datos de una orden de servicio. • <code>Public int GetNumOrden():</code> Obtener el valor del identificador de la orden. • <code>Public String GetPropietario():</code> Obtener el valor del nombre del propietario. • <code>Public String GetPlaca():</code> Obtener el valor de la placa del vehiculo. • <code>Public String GetDescripcion():</code> Obtener el valor de la descripción de la orden. • <code>Public double GetCosto():</code> Obtener el valor del costo del servicio. • <code>Public void SetDescripcion(String descripcion):</code> Para dar un valor a la descripción de la orden. • <code>Public void SetCosto(double costo):</code> Para dar un valor al costo de la orden.
ControladorOrdenes	• <code>Private List<OrdenServicio> ordenes:</code> Lista de todas las ordenes de servicio. • <code>Private VistaOrdenes vista:</code> Vista para llevar a cabo la interacción con el usuario.	• <code>Public ControladorOrdenes():</code> Constructor de la clase ControladorOrdenes. • <code>Public void Iniciar():</code> Inicializa el programa. • <code>Public void RegistrarOrden(OrdenServicio orden):</code> Añadir una orden nueva a la lista. • <code>Public List<OrdenServicio> GetOrdenes():</code> Para devolver la lista de las ordenes. • <code>Public OrdenServicio BuscarOrden(int numOrden):</code> Busca una orden mediante su número. • <code>Public void modificarDescripcion(int numOrden, String nuevaDescripcion):</code> Para buscar una orden mediante su número y modificar la descripción del servicio. • <code>Public void modificarCosto(int numOrden, double nuevoCosto):</code> Para

		buscar una orden mediante su número y modificar el costo estimado. • Public void CancelarOrden(int numOrden): Busca una orden por número y la elimina. • Public List<OrdenServicio> ConsultarPorPlaca(String placa): Busca todas las ordenes asociadas a una placa. • Public double ReporteTotal(): Calcula el valor total de todas las órdenes. • Public double ReportePromedio(): Calcula el valor promedio de todas las órdenes. • Public OrdenServicio OrdenDeCostoMayor(): Calcula cual es la orden con el costo más elevado. • Public int CantidadOrdenes(): Calcula cuantas ordenes hay actualmente.
VistaOrdenes	• Private Scanner sc: Para leer los datos que ingrese el usuario.	• Public int MostrarMenu(): Para mostrar todas las opciones y escoger una. • Public OrdenServicio LeerOrdenServicio(): Lee los datos de una nueva orden. • Public void MostrarOrdenes(List<OrdenServicio> ordenes): Muestra todas las ordenes registradas. • Public int LeerNumeroOrden(): Lee el numero de orden a buscar. • Public void MostrarOrdenBuscada(OrdenServicio orden): Para mostrar la orden buscada por su número. • Public int MostrarOpcionesModificacion(): Para mostrar las opciones de modificacion de orden. • Public String leerNuevaDescripcion(): Para solicitar y devolver la nueva descripción del servicio. • Public double leerNuevoCosto(): Para solicitar y devolver el nuevo costo. • Public void OrdenCancelada(): Muestra un mensaje de que la orden fue cancelada. • Public String LeerPlaca(): Para leer la placa que desea ingresar el usuario. • Public void MostrarOrdenConsultadaPlaca(List<Or

		denServicio> ordenes): Para mostrar las ordenes buscadas por placa. • Public void MostrarReporteCostos(double total, double promedio): Muestra el valor total y el promedio de todas las órdenes. • Public void MostrarOrdenDeCostoMayor(OrdenServicio orden): Muestra la orden con el costo mas elevado. • Public void MostrarCantidadOrdenes(int cantidad): Muestra cuantas ordenes hay actualmente.
--	--	--

4. ¿Qué información deberá almacenarse dentro de la colección dinámica?

La colección dinámica almacenará objetos de tipo OrdenServicio. Cada objeto contendrá el número de orden, el nombre del propietario, la placa del vehículo, la descripción del servicio y el costo estimado. La colección guardará las referencias a todas las órdenes registradas actualmente.

5. ¿Dónde utilizará List y dónde realizará la creación mediante ArrayList?

En ControladorOrdenes se declarará la propiedad ordenes utilizando List<OrdenServicio>. La lista se creará en el constructor de la clase mediante new ArrayList<OrdenServicio>(). List será el tipo declarado y ArrayList será la implementación del arreglo dinámico.

8. ¿Cómo proveerá de valores iniciales a sus objetos? ¿Qué valores iniciales les asignará?

Los objetos de tipo OrdenServicio recibirán sus valores iniciales mediante su constructor, estos valores serán solicitados al usuario: número de orden, propietario, placa, descripción y costo estimado. La lista de órdenes del controlador se inicializará como un ArrayList vacío, ya que al comenzar el programa todavía no existirán órdenes registradas.

9. ¿Cómo identificará una orden específica dentro de la colección?

Cada orden será identificada mediante su número de orden, el cual deberá ser único. Para localizarla, se recorrerá la colección y se comparará el número buscado con el valor devuelto por getNumOrden() para cada orden. Si se encuentra una coincidencia, se devolverá la referencia al objeto correspondiente y si no existe, se producirá una excepción.

10. ¿Cómo agregará y eliminará órdenes de la colección dinámica?

Para agregar una orden, primero se creará un objeto `OrdenServicio` con los datos ingresados por el usuario y después se utilizará el método `add()` de la lista. Antes de agregarlo, se comprobará que su número de orden no esté registrado.

Para eliminar una orden, primero se buscará mediante su número. El objeto encontrado será una referencia al mismo objeto almacenado en la lista, por lo que podrá eliminarse utilizando `remove()`. Si la orden no existe, se producirá una excepción y la colección no será modificada.

11. ¿Cómo realizará la búsqueda de todas las órdenes asociadas a una misma placa?

Se recorrerá la lista principal de órdenes y se comparará la placa de cada objeto con la placa ingresada por el usuario mediante `equals()`. Cada coincidencia se agregará a una lista de tipo `List<OrdenServicio>`, creada como `ArrayList`. Finalmente, esta lista será devuelta a la vista para mostrar todas las órdenes encontradas. Si la lista temporal está vacía, se mostrará que no existen órdenes asociadas a esa placa.

12. ¿Cómo recorrerá la colección para calcular el valor total y el costo promedio de las órdenes?

La lista se recorrerá con un ciclo for-each. En cada iteración se obtendrá el costo de la orden y se acumulará en una variable double, el valor total será el resultado de la suma de todos los costos. Para calcular el promedio, el total se dividirá dentro de la cantidad de elementos obtenida mediante size().

13. ¿Cómo determinará cuál es la orden con el costo estimado más alto?

Se recorrerá la lista comparando el costo de cada orden y se guardará una referencia a la orden con el costo más alto encontrado hasta ese momento. Cuando se encuentre una orden con un costo mayor, se reemplazará la referencia almacenada. Al terminar el recorrido, se devolverá la orden de mayor costo.

14. ¿Qué situaciones dentro del programa pueden producir una excepción?

- Ingresar texto cuando se espera un número.
- Ingresar un costo menor o igual que cero.
- Ingresar vacío el nombre del propietario, la placa o la descripción.
- Intentar registrar un número de orden que ya existe.
- Buscar una orden que no está registrada.
- Modificar una orden que no existe.
- Cancelar una orden que no existe.
- Solicitar la orden de mayor costo cuando la lista está vacía.
- Ingresar una opción inexistente en el menú.

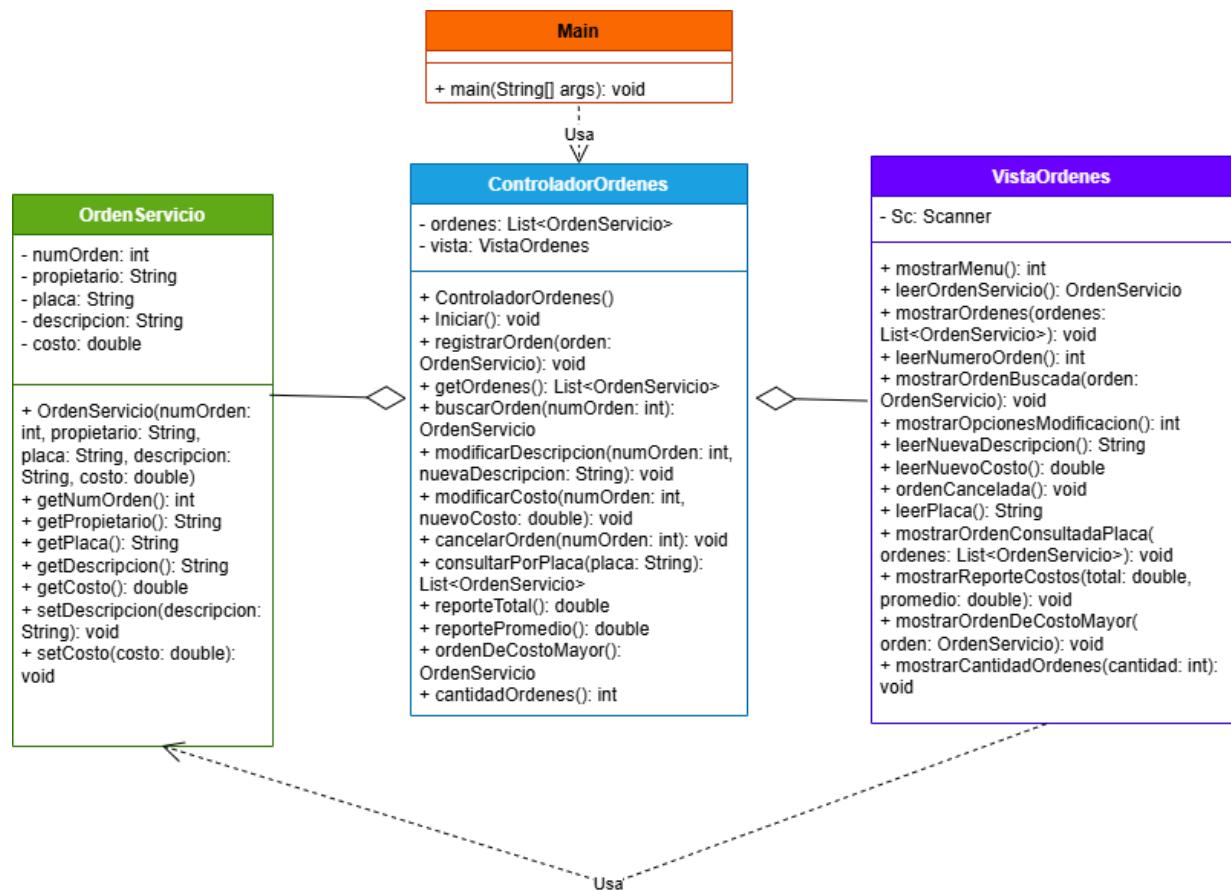
15. ¿Qué operaciones serán protegidas utilizando try-catch?

Se protegerá la lectura de opciones, números de orden y costos para controlar entradas que no puedan convertirse al tipo requerido, el registro de órdenes con datos inválidos o números repetidos y las operaciones de búsqueda, modificación y cancelación cuando el número leído no corresponda a una orden registrada.

16. ¿Qué acción realizará mediante el bloque finally y por qué debe ejecutarse independientemente del resultado de la operación?

En el proceso de búsqueda se usará un bloque finally para mostrar un mensaje indicando que la operación de búsqueda ha finalizado y que el programa regresará al menú principal. Esta acción deberá ejecutarse tanto si la orden fue encontrada como si ocurrió una excepción.

Diseño



Naranja: Driver program

Azul: Controlador

Morado: Vista

Verde: Modelo